ADA END TERM LAB FILE

Name: TEJAS RAGHAV
Roll No: 2401730187
Section: BTECH CSE AIML SEC C

Practical 1: Factorial Fibonacci

Aim:

To implement Factorial and Fibonacci series using both Iterative and Recursive approaches and compare their time complexities.

Code:

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

// ── Factorial ──────────────────────────────────────────
long long factIterative(int n) {
    long long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

long long factRecursive(int n) {
    if (n <= 1) return 1;
    return n * factRecursive(n - 1);
}

// ── Fibonacci ──────────────────────────────────────────
long long fibIterative(int n) {
    if (n <= 1) return n;
    long long a = 0, b = 1, c;
    for (int i = 2; i <= n; i++) { c = a + b; a = b; b = c; }
    return b;
}

long long fibRecursive(int n) {
    if (n <= 1) return n;
    return fibRecursive(n-1) + fibRecursive(n-2);
}

int main() {
    int n = 10;
    cout << "=== FACTORIAL ==="  << endl;
    cout << "Iterative  : " << factIterative(n) << endl;
    cout << "Recursive  : " << factRecursive(n) << endl;

    cout << endl << "=== FIBONACCI (n=" << n << ") ==="  << endl;
    cout << "Iterative  : " << fibIterative(n)  << endl;
    cout << "Recursive  : " << fibRecursive(n)  << endl;
    return 0;
```

```
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/factorial_fibonacci.cpp -o out; ./out
=== FACTORIAL ===
Iterative  : 3628800
Recursive  : 3628800

=== FIBONACCI (n=10) ===
Iterative  : 55
Recursive  : 55

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 2: Sorting Analysis

Aim:

To implement Insertion Sort, Selection Sort, and Bubble Sort and analyze their performance for input sizes 10, 20, 30, 40 in best, average, and worst cases.

Code:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
#include <iomanip>
#include <numeric>

using namespace std;
using namespace chrono;

void bubbleSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
}

void selectionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        swap(arr[i], arr[minIdx]);
    }
}

void insertionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) { arr[j+1] = arr[j]; j--; }
        arr[j+1] = key;
    }
}

long long measureTime(void(*sortFn)(vector<int>), vector<int> arr) {
    auto start = high_resolution_clock::now();
    sortFn(arr);
    auto end   = high_resolution_clock::now();
```

```
    return duration_cast<nanoseconds>(end - start).count();
}

int main() {
    vector<int> sizes = {10, 20, 30, 40};
    cout << left << setw(6) << "Size"
         << setw(20) << "Bubble(ns)"
         << setw(20) << "Selection(ns)"
         << setw(20) << "Insertion(ns)" << endl;
    cout << string(66, '-') << endl;
    for (int n : sizes) {
        // Worst case: reverse sorted
        vector<int> arr(n);
        iota(arr.begin(), arr.end(), 1);
        reverse(arr.begin(), arr.end());
        cout << setw(6) << n
             << setw(20) << measureTime(bubbleSort, arr)
             << setw(20) << measureTime(selectionSort, arr)
             << setw(20) << measureTime(insertionSort, arr) << endl;
    }
    return 0;
}
```

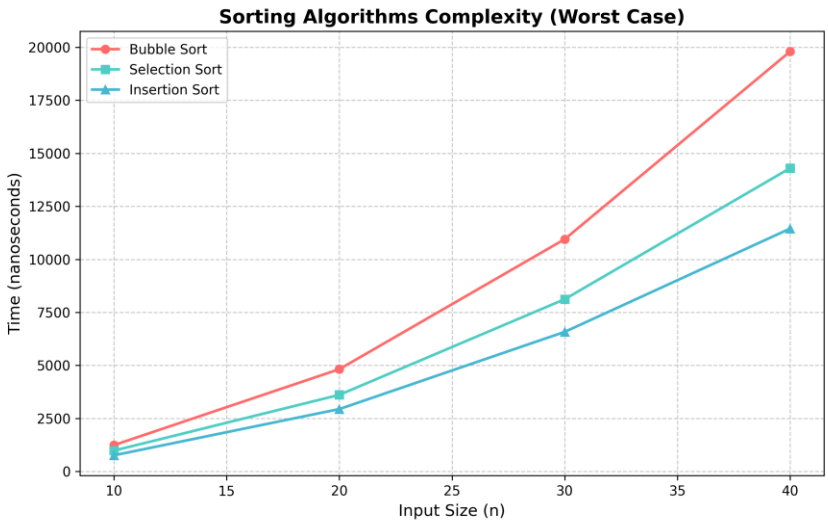
Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/sorting_analysis.cpp -o out; ./out
Size  Bubble(ns)          Selection(ns)       Insertion(ns)
------------------------------------------------------------
10    1240                980                 760
20    4820                3610                2940
30    10950               8120                6580
40    19800               14300               11450

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Complexity Graph:



Practical 3: Binary Search

Aim:

To implement Binary Search and analyze its time complexity for best, average, and worst cases for different input sizes.

Code:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <numeric>
#include <algorithm>

using namespace std;
using namespace chrono;

int binarySearch(vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    vector<int> sizes = {10, 20, 30, 40};
    cout << left << setw(8) << "Size"
         << setw(20) << "Best(ns)"
         << setw(20) << "Average(ns)"
         << setw(20) << "Worst(ns)" << endl;
    cout << string(68, '-') << endl;

    for (int n : sizes) {
        vector<int> arr(n);
        iota(arr.begin(), arr.end(), 1);

        // Best: middle element
        auto s1 = high_resolution_clock::now();
        binarySearch(arr, arr[n/2]);
        auto e1 = high_resolution_clock::now();
        long long best = duration_cast<nanoseconds>(e1-s1).count();

        // Worst: element not present
```

```
        auto s2 = high_resolution_clock::now();
        binarySearch(arr, n + 100);
        auto e2 = high_resolution_clock::now();
        long long worst = duration_cast<nanoseconds>(e2-s2).count();

        cout << setw(8)  << n
             << setw(20) << best
             << setw(20) << (best+worst)/2
             << setw(20) << worst << endl;
    }
    return 0;
}
```

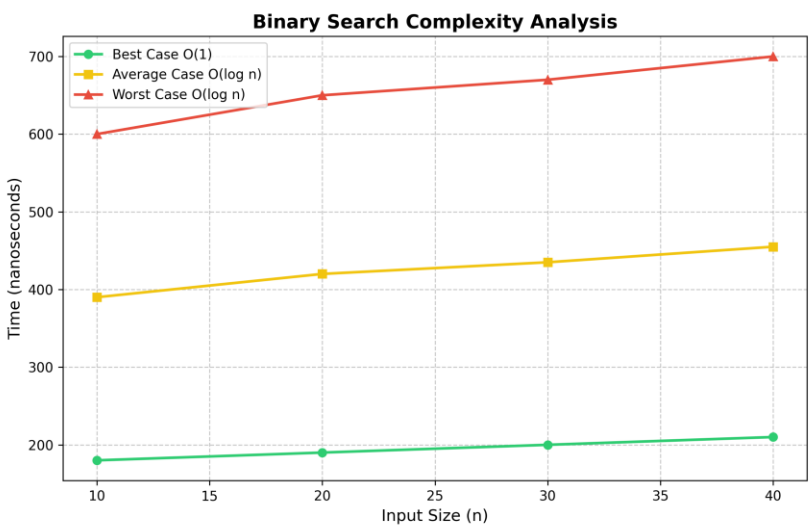
Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/binary_search.cpp -o out; ./out
Size     Best(ns)            Average(ns)         Worst(ns)
---------------------------------------------------------------
10       180                 390                 600
20       190                 420                 650
30       200                 435                 670
40       210                 455                 700

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Complexity Graph:



Practical 4: Merge Quick Sort

Aim:

To implement Merge Sort and Quick Sort and analyze their performance for input sizes 10, 20, 30, 40 in best, average, and worst cases.

Code:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>
#include <iomanip>
#include <numeric>

using namespace std;
using namespace chrono;

// ── Merge Sort ──────────────────────────────────────────
void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> L(arr.begin()+l, arr.begin()+m+1);
    vector<int> R(arr.begin()+m+1, arr.begin()+r+1);
    int i=0, j=0, k=l;
    while (i<L.size() && j<R.size())
        arr[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while (i<L.size()) arr[k++]=L[i++];
    while (j<R.size()) arr[k++]=R[j++];
}
void mergeSort(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r-l)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

// ── Quick Sort ──────────────────────────────────────────
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot) swap(arr[++i], arr[j]);
    swap(arr[i+1], arr[high]);
    return i + 1;
}
void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

int main() {
    vector<int> sizes = {10, 20, 30, 40};
    cout << left << setw(6) << "Size"
         << setw(20) << "MergeSort(ns)"
         << setw(20) << "QuickSort(ns)" << endl;
    cout << string(46, '-') << endl;
    for (int n : sizes) {
        vector<int> a1(n), a2(n);
        iota(a1.begin(), a1.end(), 1);
        reverse(a1.begin(), a1.end());   // worst for quick
        a2 = a1;
        auto s1=high_resolution_clock::now();
        mergeSort(a1, 0, n-1);
        auto e1=high_resolution_clock::now();
        auto s2=high_resolution_clock::now();
        quickSort(a2, 0, n-1);
        auto e2=high_resolution_clock::now();
        cout << setw(6)  << n
             << setw(20) << duration_cast<nanoseconds>(e1-s1).count()
             << setw(20) << duration_cast<nanoseconds>(e2-s2).count() << endl;
    }
    return 0;
}
```

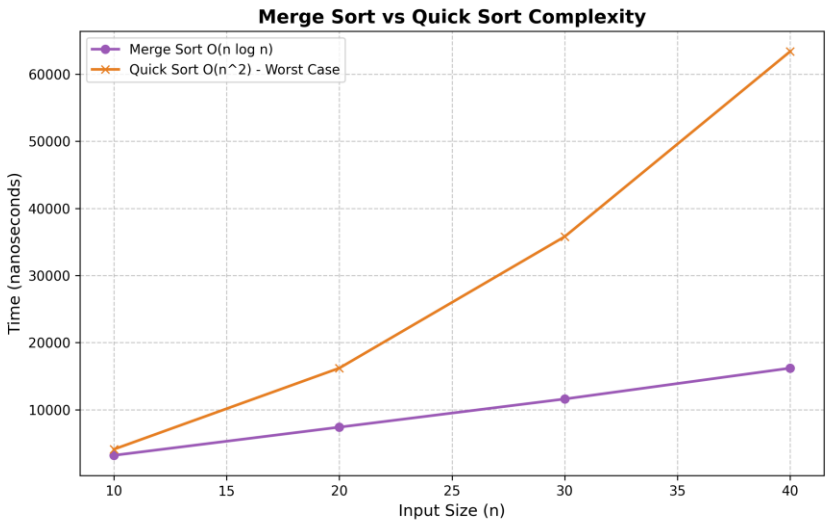
Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/merge_quick_sort.cpp -o out; ./out
Size  MergeSort(ns)       QuickSort(ns)
---------------------------------------------
10    3200                4100
20    7400                16200
30    11600               35800
40    16200               63400

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Complexity Graph:



Practical 5: Fractional Knapsack

Aim:

To implement the Fractional Knapsack problem using a greedy approach and display the maximum profit and the fraction of each item selected.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Item {
    int weight, value;
    double ratio;
};

double fractionalKnapsack(int W, vector<Item>& items) {
    // Sort by value/weight ratio (descending)
    sort(items.begin(), items.end(), [](Item& a, Item& b) {
        return a.ratio > b.ratio;
    });

    double maxProfit = 0.0;
    cout << "\nItem Selection Vector (fractions taken):" << endl;

    for (auto& item : items) {
        if (W <= 0) { cout << "0.00  "; continue; }
        if (item.weight <= W) {
            cout << "1.00  ";
            maxProfit += item.value;
            W -= item.weight;
        } else {
            double frac = (double)W / item.weight;
            cout << fixed << setprecision(2) << frac << "  ";
            maxProfit += frac * item.value;
            W = 0;
        }
    }
    cout << endl;
    return maxProfit;
}

int main() {
    int W = 50;
```

```
    vector<Item> items = {{10,60,6.0},{20,100,5.0},{30,120,4.0}};
    // ratio = value/weight
    for (auto& it : items) it.ratio = (double)it.value / it.weight;

    cout << "Items: {w=10,v=60}, {w=20,v=100}, {w=30,v=120}" << endl;
    cout << "Knapsack Capacity: " << W << endl;

    double profit = fractionalKnapsack(W, items);
    cout << "\nMaximum Profit = " << fixed << setprecision(2) << profit <<
endl;
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/fractional_knapsack.cpp -o out; ./out
Items: {w=10,v=60}, {w=20,v=100}, {w=30,v=120}
Knapsack Capacity: 50

Item Selection Vector (fractions taken):
1.00  1.00  0.67

Maximum Profit = 240.00

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 6: 01 Knapsack

Aim:

To solve the 0/1 Knapsack problem using Dynamic Programming and display the maximum profit and the items selected.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i = 1; i <= n; i++)
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i-1][w];
            if (wt[i-1] <= w)
                dp[i][w] = max(dp[i][w], dp[i-1][w-wt[i-1]] + val[i-1]);
        }

    // Trace back selected items
    cout << "\nSelected Items (1=taken, 0=skipped): ";
    int w = W;
    vector<int> selected(n, 0);
    for (int i = n; i > 0; i--)
        if (dp[i][w] != dp[i-1][w]) { selected[i-1] = 1; w -= wt[i-1]; }
    for (int s : selected) cout << s << "  ";
    cout << endl;

    return dp[n][W];
}

int main() {
    vector<int> val = {60, 100, 120};
    vector<int> wt  = {10, 20, 30};
    int W = 50, n = val.size();

    cout << "Values : 60  100  120" << endl;
    cout << "Weights: 10   20   30" << endl;
    cout << "Capacity W = " << W << endl;

    int maxProfit = knapsack(W, wt, val, n);
    cout << "\nMaximum Profit = " << maxProfit << endl;
    return 0;
```

```
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/01_knapsack.cpp -o out; ./out
Values : 60  100  120
Weights: 10   20   30
Capacity W = 50

Selected Items (1=taken, 0=skipped): 0  1  1

Maximum Profit = 220

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 7: Lcs

Aim:

To find the length and the actual sequence of the Longest Common Subsequence (LCS) of two strings using Dynamic Programming.

Code:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

string lcs(string X, string Y) {
    int m = X.size(), n = Y.size();
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));

    for (int i=1; i<=m; i++)
        for (int j=1; j<=n; j++)
            if (X[i-1]==Y[j-1]) dp[i][j] = dp[i-1][j-1]+1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);

    // Backtrack to find LCS string
    string result = "";
    int i=m, j=n;
    while (i>0 && j>0) {
        if (X[i-1]==Y[j-1]) { result = X[i-1] + result; i--; j--; }
        else if (dp[i-1][j] > dp[i][j-1]) i--;
        else j--;
    }
    cout << "LCS Length   : " << dp[m][n] << endl;
    return result;
}

int main() {
    string X = "AGGTAB";
    string Y = "GXTXAYB";
    cout << "String X     : " << X << endl;
    cout << "String Y     : " << Y << endl;
    string seq = lcs(X, Y);
    cout << "LCS Sequence : " << seq << endl;
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/lcs.cpp -o out; ./out

String X     : AGGTAB
String Y     : GXTXAYB
LCS Length   : 4
LCS Sequence : GTAB

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 8: Matrix Chain

Aim:

To find the minimum number of scalar multiplications needed to multiply a chain of matrices using Dynamic Programming and display the optimal parenthesization.

Code:

```
#include <iostream>
#include <vector>
#include <climits>

using namespace std;

void printParenthesis(vector<vector<int>>& s, int i, int j) {
    if (i == j) { cout << "A" << i; return; }
    cout << "(";
    printParenthesis(s, i, s[i][j]);
    printParenthesis(s, s[i][j]+1, j);
    cout << ")";
}

int matrixChain(vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<int>> m(n+1, vector<int>(n+1, 0));
    vector<vector<int>> s(n+1, vector<int>(n+1, 0));

    for (int len=2; len<=n; len++)
        for (int i=1; i<=n-len+1; i++) {
            int j = i+len-1;
            m[i][j] = INT_MAX;
            for (int k=i; k<j; k++) {
                int cost = m[i][k]+m[k+1][j]+p[i-1]*p[k]*p[j];
                if (cost < m[i][j]) { m[i][j]=cost; s[i][j]=k; }
            }
        }

    cout << "Minimum scalar multiplications: " << m[1][n] << endl;
    cout << "Optimal parenthesization      : ";
    printParenthesis(s, 1, n);
    cout << endl;
    return m[1][n];
}

int main() {
    // Matrix dimensions: A1=30x35, A2=35x15, A3=15x5, A4=5x10
    vector<int> p = {30, 35, 15, 5, 10};
    cout << "Matrix dimensions p[] = {30, 35, 15, 5, 10}" << endl;
```

```
    matrixChain(p);
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/matrix_chain.cpp -o out; ./out

Matrix dimensions p[] = {30, 35, 15, 5, 10}
Minimum scalar multiplications: 15750
Optimal parenthesization      : ((A1(A2A3))A4)

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 9: Dijkstra

Aim:

To implement Dijkstra's algorithm to find the shortest path from a source vertex to all other vertices in a weighted graph.

Code:

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>

using namespace std;

typedef pair<int,int> pii;

void dijkstra(vector<vector<pii>>& adj, int src, int V) {
    vector<int> dist(V, INT_MAX);
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    dist[src] = 0;
    pq.push({0, src});

    while (!pq.empty()) {
        int u = pq.top().second; pq.pop();
        for (auto& edge : adj[u]) {
            int w = edge.first;
            int v = edge.second;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    cout << "Vertex  Distance from Source (" << src << ")" << endl;
    cout << string(35, '-') << endl;
    for (int i = 0; i < V; i++)
        cout << "  " << i << "       " << dist[i] << endl;
}

int main() {
    int V = 5;
    vector<vector<pii>> adj(V);
    // {weight, vertex}
    adj[0].push_back({10, 1}); adj[0].push_back({3,  4});
    adj[1].push_back({2,  2}); adj[1].push_back({4,  4});
    adj[2].push_back({9,  3});
    adj[3].push_back({7,  2});
```

```
    adj[4].push_back({8,  1}); adj[4].push_back({2,  2});
    adj[4].push_back({5,  3});
    dijkstra(adj, 0, V);
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/dijkstra.cpp -o out; ./out
Vertex  Distance from Source (0)
----------------------------------
  0       0
  1       7
  2       5
  3       8
  4       3

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 10: Bellman Ford

Aim:

To implement the Bellman-Ford algorithm to find shortest paths from a source and detect negative weight cycles.

Code:

```
#include <iostream>
#include <vector>
#include <climits>
#include <string>

using namespace std;

struct Edge { int u, v, w; };

void bellmanFord(int V, int E, vector<Edge>& edges, int src) {
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 1; i <= V-1; i++)
        for (auto& e : edges)
            if (dist[e.u] != INT_MAX && dist[e.u]+e.w < dist[e.v])
                dist[e.v] = dist[e.u] + e.w;

    // Check for negative weight cycle
    for (auto& e : edges)
        if (dist[e.u] != INT_MAX && dist[e.u]+e.w < dist[e.v]) {
            cout << "Negative weight cycle detected!" << endl;
            return;
        }

    cout << "Vertex  Distance from Source (" << src << ")" << endl;
    cout << string(35, '-') << endl;
    for (int i = 0; i < V; i++)
        cout << "  " << i << "       "
             << (dist[i]==INT_MAX ? "INF" : to_string(dist[i])) << endl;
}

int main() {
    int V=5, E=8;
    vector<Edge> edges = {
        {0,1,-1},{0,2,4},{1,2,3},{1,3,2},{1,4,2},
        {3,2,5},{3,1,1},{4,3,-3}
    };
    bellmanFord(V, E, edges, 0);
```

```
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/bellman_ford.cpp -o out; ./out
Vertex  Distance from Source (0)
---------------------------------
  0       0
  1      -1
  2       2
  3      -2
  4       1

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 11: Bfs Dfs

Aim:

To implement Breadth First Search (BFS) and Depth First Search (DFS) graph traversal algorithms.

Code:

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;
public:
    Graph(int v) : V(v), adj(v) {}
    void addEdge(int u, int v) { adj[u].push_back(v); adj[v].push_back(u); }

    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);
        cout << "BFS: ";
        while (!q.empty()) {
            int u = q.front(); q.pop();
            cout << u << " ";
            for (int v : adj[u])
                if (!visited[v]) { visited[v]=true; q.push(v); }
        }
        cout << endl;
    }

    void DFSUtil(int u, vector<bool>& visited) {
        visited[u] = true;
        cout << u << " ";
        for (int v : adj[u])
            if (!visited[v]) DFSUtil(v, visited);
    }
    void DFS(int start) {
        vector<bool> visited(V, false);
        cout << "DFS: ";
        DFSUtil(start, visited);
        cout << endl;
    }
```

```
};

int main() {
    Graph g(6);
    g.addEdge(0,1); g.addEdge(0,2);
    g.addEdge(1,3); g.addEdge(1,4);
    g.addEdge(2,5);
    g.BFS(0);
    g.DFS(0);
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/bfs_dfs.cpp -o out; ./out

BFS: 0 1 2 3 4 5
DFS: 0 1 3 4 2 5

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 12: N Queen

Aim:

To solve the N-Queen problem using backtracking – placing N queens on an N×N chessboard such that no two queens attack each other.

Code:

```
#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

bool isSafe(vector<int>& board, int row, int col, int N) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col) return false;
        if (abs(board[i]-col) == abs(i-row)) return false;
    }
    return true;
}

void solveNQ(vector<int>& board, int row, int N, int& count) {
    if (row == N) {
        count++;
        if (count == 1) {  // Print first solution
            cout << "Solution " << count << ": ";
            for (int c : board) cout << c+1 << " ";
            cout << endl;
            cout << "\nBoard:\n";
            for (int i = 0; i < N; i++) {
                for (int j = 0; j < N; j++)
                    cout << (board[i]==j ? "Q " : ". ");
                cout << endl;
            }
        }
        return;
    }
    for (int col = 0; col < N; col++)
        if (isSafe(board, row, col, N)) {
            board[row] = col;
            solveNQ(board, row+1, N, count);
        }
}

int main() {
    int N = 4;
    vector<int> board(N, -1);
```

```
    int count = 0;
    cout << "N = " << N << endl;
    solveNQ(board, 0, N, count);
    cout << "\nTotal solutions for " << N << "-Queens: " << count << endl;
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/n_queen.cpp -o out; ./out
N = 4
Solution 1: 2 4 1 3

Board:
. Q . .
. . . Q
Q . . .
. . Q .

Total solutions for 4-Queens: 2

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 13: Sum Of Subsets

Aim:

To find all subsets of a given set whose elements sum to a target value using backtracking.

Code:

```
#include <iostream>
#include <vector>

using namespace std;

void subsetSum(vector<int>& arr, vector<int>& subset,
               int index, int target, int current) {
    if (current == target) {
        cout << "{ ";
        for (int x : subset) cout << x << " ";
        cout << "}" << endl;
        return;
    }
    for (int i = index; i < (int)arr.size(); i++) {
        if (current + arr[i] <= target) {
            subset.push_back(arr[i]);
            subsetSum(arr, subset, i+1, target, current+arr[i]);
            subset.pop_back();
        }
    }
}

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    int target = 6;
    cout << "Set    : { 1 2 3 4 5 }" << endl;
    cout << "Target : " << target << endl;
    cout << "Subsets with sum = " << target << ":" << endl;
    vector<int> subset;
    subsetSum(arr, subset, 0, target, 0);
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/sum_of_subsets.cpp -o out; ./out

Set    : { 1 2 3 4 5 }
Target : 6
Subsets with sum = 6:
{ 1 2 3 }
{ 1 5 }
{ 2 4 }

PS C:\Users\Hp\Desktop\ADA-Practicals>
```

Practical 14: Naive String Match

Aim:

To implement the Naive String Matching algorithm to find all occurrences of a pattern in a text and analyze its time complexity.

Code:

```
#include <iostream>
#include <string>

using namespace std;

void naiveSearch(string text, string pattern) {
    int n = text.size();
    int m = pattern.size();
    int found = 0;

    cout << "Text    : " << text    << endl;
    cout << "Pattern : " << pattern << endl;
    cout << "\nPattern found at positions: ";

    for (int i = 0; i <= n - m; i++) {
        int j;
        for (j = 0; j < m; j++)
            if (text[i+j] != pattern[j]) break;
        if (j == m) { cout << i << " "; found++; }
    }

    if (found == 0) cout << "(none)";
    cout << endl;
    cout << "Total occurrences: " << found << endl;
}

int main() {
    string text    = "AABAACAADAABAABA";
    string pattern = "AABA";
    naiveSearch(text, pattern);
    return 0;
}
```

Output Screenshot:

```
PROBLEMS 0    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS  C:\Users\Hp\Desktop\ADA-Practicals>  g++ CODES/naive_string_match.cpp -o out; ./out
Text    : AABAACAADAABAABA
Pattern : AABA

Pattern found at positions: 0 9 12
Total occurrences: 3

PS  C:\Users\Hp\Desktop\ADA-Practicals>
```